
Indice

1. Marco Referencial	1
1.1. Introducción	1
1.2. Antecedentes	2
1.2.1. Antecedentes del objeto de estudio	2
1.2.2. Referencias técnicas de otros trabajos	3
1.2.3. Descripción del contexto	5
1.3. Descripción del objeto de estudio	5
1.4. Identificación del Problema	6
1.5. Formulación del Problema	7
1.6. Objetivos	8
1.6.1. Objetivo General	8
1.6.2. Objetivos Específicos	8
1.7. Justificaciones	9
1.7.1. Justificación técnica	9
1.7.2. Justificación social	9
1.7.3. Justificación económica	9
1.8. Límites y Alcances	10
1.8.1. Límites	10
1.8.2. Alcances	10
1.9. Metodología de la investigación	11
1.9.1. Tipo de estudio	11
1.9.2. Métodos	11
1.9.3. Técnicas	12
1.10. Análisis preliminar	12

1.11. Propuesta de solución	12
1.12. Modelos y herramientas a utilizar	13
1.13. Cronograma	15
2. Marco Teórico	16
2.1. Metodología de análisis y diseño	16
2.2. Metodología de desarrollo	16
2.3. Fundamentos de Inteligencia Artificial	17
2.3.1. Modelos de Lenguaje de Gran Escala (LLM)	17
2.3.2. Alucinación Artificial	17
2.3.3. Generación Aumentada por Recuperación (RAG)	18
2.3.4. Graph RAG: Recuperación Aumentada sobre Grafo	18
2.3.5. Embeddings Semánticos y Bases de Datos Vectoriales	19
2.3.6. Bases de Datos de Grafos: Neo4j	19
2.3.7. LangGraph y Agentes de Múltiples Estados	19
2.3.8. Proceso ETL y Web Scraping	20
2.4. Fundamentos Jurídicos	20
2.4.1. LegalTech y Acceso a la Justicia	20
2.4.2. Marco Normativo Boliviano	20
2.5. Tecnologías de Soporte	21
2.5.1. FastAPI	21
2.5.2. Next.js 14	21
2.5.3. Docker y Contenedorización	22
2.5.4. Infraestructura en la Nube	22
3. Marco Práctico	23
3.1. Análisis preliminar	23
3.2. Determinación de requerimientos	24
3.2.1. Requerimientos funcionales	24
3.2.2. Requerimientos no funcionales	25
3.3. Análisis del proyecto	26
3.3.1. Diseño UML	26
3.4. Diseño del proyecto	34
3.4.1. Arquitectura de Software (Modelo C4)	34
3.4.2. Diseño de Experiencia y Prototipado (UX/UI)	43
3.5. Base de datos	43
3.6. Validación y pruebas del sistema	44

3.7. Análisis de costos	45
3.8. Desarrollo del Prototipo	45
4. Conclusiones y Recomendaciones	47
4.1. Conclusiones	47
4.2. Recomendaciones	48
Referencias Bibliográficas	49

Indice de figuras

1.1. Interfaz de la aplicación DoNotPay.	3
1.2. LawBot, sistema de orientación legal desarrollado en Cambridge.	4
1.3. Diagrama de Causa-Efecto (Ishikawa) sobre la brecha de acceso a la justicia.	7
1.4. Modelo conceptual de la transformación del acceso a la información legal.	13
1.5. Diagrama de Gantt: Cronograma de implementación para Taller de Gra- do II.	15
2.1. Diagrama del proceso ETL para la normativa legal.	20
2.2. Modelo de Responsabilidad Compartida en la Nube.	22
3.1. Diagrama del pipeline de procesamiento de consultas legales (Graph RAG).	23
3.2. Diagrama de Caso de Uso: Interacción de consulta Graph RAG.	26
3.3. Diagrama de Caso de Uso: Supervisión y Auditoría.	27
3.4. Diagrama de Caso de Uso: Mantenimiento de la Base de Conocimiento. .	28
3.5. Diagrama de Clases: Arquitectura de capas del sistema LexIA.	29
3.6. Diagrama de Secuencia: Orquestación del flujo Graph RAG con Lang- Graph.	30
3.7. Diagrama de Actividades: Interacción del usuario con LexIA.	31
3.8. Diagrama de Actividades: Algoritmo de actualización ETL con control de fallos.	32
3.9. Diagrama de Contexto (C4): Fronteras y dependencias externas de LexIA.	34
3.10. Diagrama de Contenedores (C4): Arquitectura de servicios distribuidos. .	37
3.11. Diagrama de Componentes (C4): Estructura interna del Backend API. . .	40
3.12. Diagrama de Código (C4): Diseño interno del Pipeline ETL.	42

3.13. Prototipo de UI: Interfaz principal de consulta en lenguaje natural.	43
3.14. Prototipo de UI: Presentación de la orientación legal y fuentes normativas.	43

Indice de tablas

1.1. Modelos de ingeniería aplicados al proyecto.	13
1.2. Stack tecnológico del sistema LexIA.	14
2.1. Comparación técnica: RAG vs. Fine-Tuning para dominios legales.	18
2.2. Cuerpos legales principales que conforman la base de conocimiento.	21
3.1. Especificación de Requerimientos Funcionales.	24
3.2. Especificación de Requerimientos No Funcionales.	25
3.3. Métricas RAGAS: RAG estándar vs. Graph RAG (LexIA).	44
3.4. Estimación de recursos y costos operativos mensuales.	45

CAPÍTULO 1

Marco Referencial

1.1. Introducción

El presente proyecto de grado aborda el desarrollo de **LexIA**, un sistema de orientación legal conversacional basado en inteligencia artificial, diseñado para brindar asistencia jurídica inicial a ciudadanos bolivianos que enfrentan situaciones legales en diversas áreas del derecho: penal, civil, laboral, tránsito, comercial, administrativo, constitucional y tributario. La propuesta busca democratizar el acceso a la información jurídica traduciendo el lenguaje técnico legal a un formato claro y accionable mediante el uso de lenguaje natural.

La investigación se enmarca dentro del área de la Inteligencia Artificial aplicada al Derecho (*LegalTech*), empleando técnicas avanzadas de Procesamiento de Lenguaje Natural y una arquitectura novedosa denominada *Graph RAG* (Generación Aumentada por Recuperación sobre Grafo de Conocimiento). A diferencia de los sistemas RAG convencionales que recuperan fragmentos de texto por similitud semántica, LexIA navega explícitamente las relaciones entre entidades jurídicas —qué ley deroga a otra, qué decreto reglamenta qué norma, qué artículos se remiten mutuamente— produciendo respuestas más completas y jurídicamente coherentes.

El sistema no solo tiene como objetivo recuperar artículos exactos de la normativa, sino funcionar como un agente conversacional inteligente capaz de clasificar el área

legal de la consulta, solicitar aclaraciones cuando la situación es ambigua, evaluar la complejidad del caso y, cuando esta lo requiere, derivar al ciudadano hacia abogados especializados. Este comportamiento se implementa mediante un grafo de estados de razonamiento orquestado con LangGraph, donde cada nodo representa una etapa diferenciada del procesamiento cognitivo del agente.

El estudio responde a la necesidad creciente de herramientas tecnológicas que fortalezcan la confianza en las instituciones legales del país. En este sentido, el objeto central de la investigación trasciende el desarrollo de software; se enfoca en mejorar la interacción entre la ciudadanía y el marco legal, contribuyendo a reducir la desinformación y la brecha de acceso a la justicia que afecta a amplios sectores de la población boliviana.

1.2. Antecedentes

1.2.1. Antecedentes del objeto de estudio

En el contexto nacional, la problemática del acceso a la justicia es estructural y multidimensional. Según datos oficiales, en la gestión 2023 se registraron **21.330 denuncias** por delitos contra la propiedad, cifra superior a los 20.406 casos reportados en 2022 El País, 2025. Esto equivale a un promedio de aproximadamente 57,7 denuncias diarias atendidas únicamente por la División de Propiedades de la Fuerza Especial de Lucha Contra el Crimen (FELCC).

Sin embargo, estas estadísticas oficiales reflejan una fracción mínima de la realidad. La Encuesta de Victimización (EVIC, 2023) reveló que solo el **14,1 %** de las víctimas formaliza su denuncia, mientras que el 85,9 % restante opta por no acudir a las instancias oficiales Instituto Nacional de Estadística, 2023. Esta disparidad —denominada “cifra negra” de criminalidad— evidencia una barrera profunda de acceso a la justicia, motivada en gran parte por el desconocimiento de los procedimientos legales, la complejidad del lenguaje jurídico y la saturación de los canales de atención presencial.

La problemática no se limita al ámbito penal. En materia laboral, civil y administrativa, la brecha entre los derechos formalmente reconocidos y la capacidad ciudadana de ejercerlos es igualmente profunda. Un trabajador despedido injustificadamente,

un arrendatario ante un desalojo arbitrario o un pequeño empresario que desconoce sus obligaciones tributarias enfrentan barreras informativas equivalentes. La legislación boliviana, resultado de décadas de producción normativa que incluye la Constitución Política del Estado (2009), múltiples códigos sectoriales, miles de leyes ordinarias, decretos supremos y resoluciones ministeriales, genera un volumen de información que ningún profesional puede dominar en su integridad y que resulta prácticamente inaccesible para el ciudadano sin formación jurídica.

1.2.2. Referencias técnicas de otros trabajos

A nivel internacional, el desarrollo de asistentes legales automatizados ha sentado precedentes relevantes que sirven de referencia técnica para este proyecto.

1. DoNotPay (Joshua Browder, Estados Unidos, 2015). Originalmente concebido como un chatbot para automatizar la apelación de multas de tránsito, el sistema evolucionó hacia una plataforma integral de gestión de trámites legales menores Jurafsky y Martin, 2023. Si bien DoNotPay demostró la viabilidad técnica de la automatización legal, su diseño está orientado al sistema jurídico anglosajón (*Common Law*), lo que limita su aplicabilidad al contexto boliviano, basado en Derecho Civil Continental. Además, DoNotPay no implementa un grafo de conocimiento legal ni la navegación de relaciones entre normas, capacidades centrales de LexIA.



Figura 1.1: Interfaz de la aplicación DoNotPay.

2. LawBot (Universidad de Cambridge, Reino Unido). Este sistema se orientó a brindar información legal preliminar a víctimas de delitos sexuales y otros delitos menores Estado Plurinacional de Bolivia, 1972. LawBot validó el potencial de los sistemas conversacionales para guiar a las víctimas en momentos de crisis. No obstante, el proyecto no llegó a consolidarse comercialmente y carecía de funcionalidades críticas para el contexto boliviano: integración de un marco legal completo, actualización automática de normativa y un grafo de relaciones entre normas jurídicas.



Figura 1.2: LawBot, sistema de orientación legal desarrollado en Cambridge.

3. Microsoft Graph RAG (Microsoft Research, 2024). Edge et al. (2024) propusieron una arquitectura que extiende el paradigma RAG mediante la construcción de grafos de conocimiento a partir de corpus documentales. Si bien la investigación se centró en dominios genéricos, demostró que la combinación de búsqueda vectorial con navegación de grafo produce respuestas más completas y contextualizadas. LexIA adapta esta arquitectura específicamente al dominio jurídico boliviano, con un grafo que modela las relaciones reales entre leyes, decretos, artículos, áreas del derecho y procesos legales.

A diferencia de estos antecedentes, LexIA se diferencia por: (a) su adaptación específica a la legislación boliviana vigente obtenida automáticamente de la Gaceta Oficial; (b) la implementación de un grafo de conocimiento legal que modela relaciones entre normas (derogación, reglamentación, referencia cruzada); (c) un agente con múltiples estados de razonamiento capaz de clasificar, clarificar y evaluar la complejidad de cada caso; y (d) un sistema de derivación a abogados especializados cuando la complejidad lo requiere.

1.2.3. Descripción del contexto

El proyecto se desarrolla en el contexto boliviano, caracterizado por una notable asimetría en el acceso a la información jurídica. A pesar de que el Estado ha promulgado normativas de Gobierno Electrónico y Transparencia, la interfaz entre la legislación y la ciudadanía sigue siendo deficiente.

En el ámbito social, existe un desconocimiento generalizado sobre los procedimientos formales, lo que incide directamente en la baja tasa de denuncias (14,1 %) identificada en la Encuesta de Victimización Instituto Nacional de Estadística, 2023. En el ámbito tecnológico, la adopción de herramientas de inteligencia artificial en el sector justicia es incipiente en Bolivia. Las soluciones actuales son predominantemente repositorios estáticos de leyes (PDFs escaneados o digitales) que no permiten una interacción semántica ni la navegación de relaciones entre normas.

La Gaceta Oficial del Estado Plurinacional de Bolivia Asamblea Legislativa Plurinacional, 2024, fuente primaria de toda la normativa legal del país, opera bajo el sistema de gestión de contenidos Drupal 10 con renderizado del lado del servidor, lo que permite la extracción automatizada de datos sin necesidad de navegadores headless. Todos los documentos publicados son PDFs con texto digital incrustado, facilitando su procesamiento computacional sin recurrir a reconocimiento óptico de caracteres (OCR). Estas características técnicas de la fuente oficial han sido determinantes en las decisiones de diseño del pipeline ETL de LexIA.

1.3. Descripción del objeto de estudio

El objeto de estudio se define como un **Sistema Inteligente de Orientación Legal Conversacional basado en Graph RAG y Agentes de Múltiples Estados**. Se trata de un artefacto de software con arquitectura web que integra cuatro capacidades fundamentales:

1. **Ingestión automatizada de normativa:** Pipeline ETL orquestado con Prefect 3 que extrae, procesa e indexa la legislación vigente desde la Gaceta Oficial de Bolivia, manteniéndose actualizado de forma diaria sin intervención manual.
2. **Grafo de conocimiento legal:** Representación explícita de las relaciones entre

entidades jurídicas (leyes, decretos, artículos, áreas del derecho, procesos legales) en Neo4j, que permite la navegación contextual de la normativa.

3. **Recuperación híbrida:** Motor que combina búsqueda semántica vectorial (BGE-M3 + LanceDB) con expansión por grafo (Neo4j) y reranking mediante Cross-Encoder, produciendo un contexto normativo más completo y preciso que los enfoques vectoriales puros.
4. **Agente conversacional con múltiples estados:** Implementado con LangGraph sobre Gemini 2.5 Flash, el agente gestiona el diálogo a través de estados de clasificación, clarificación, recuperación, evaluación y respuesta, adaptando su comportamiento a la complejidad del caso.

El sistema no pretende sustituir la asesoría legal profesional, sino fungir como una herramienta de orientación procesal primaria que reduce la incertidumbre inicial del ciudadano, le informa sobre sus derechos y, cuando la complejidad del caso lo amerita, lo deriva hacia profesionales especializados.

1.4. Identificación del Problema

El acceso a la justicia en Bolivia enfrenta barreras estructurales de naturaleza informativa, económica y burocrática. La brecha entre la incidencia real de conflictos jurídicos y los casos efectivamente atendidos por el sistema legal evidencia una desconexión profunda entre la ciudadanía y el aparato normativo.

Para analizar las causas raíz de este fenómeno, se ha elaborado un Diagrama de Ishikawa (Causa-Efecto), ilustrado en la Figura 1.3.

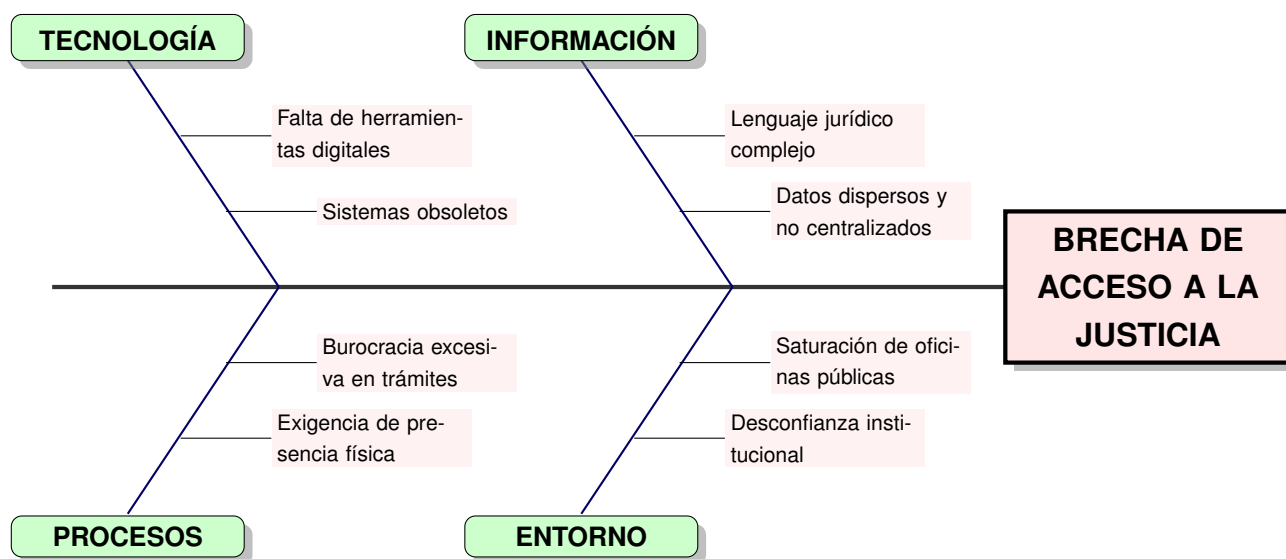


Figura 1.3: Diagrama de Causa-Efecto (Ishikawa) sobre la brecha de acceso a la justicia.

El análisis permite identificar cuatro categorías causales principales:

- **Barrera semántica:** El lenguaje técnico de la normativa resulta inaccesible para el ciudadano promedio, quien no logra traducir su experiencia vivida a términos jurídicos aplicables.
- **Obsolescencia y dispersión:** Las leyes son modificadas frecuentemente. Los ciudadanos suelen acceder a documentos desactualizados o fragmentarios en internet, generando desinformación.
- **Saturación de canales presenciales:** La demanda excede la capacidad de atención de las instituciones públicas, dejando a los ciudadanos sin orientación preliminar.
- **Carencia de herramientas contextuales:** Los modelos de lenguaje comerciales (ChatGPT, Gemini) carecen de conexión directa y verificada con la normativa boliviana vigente, propiciando respuestas incorrectas o “alucinaciones” legales.

1.5. Formulación del Problema

A partir del diagnóstico anterior, se formula la siguiente interrogante de investigación:

¿Cómo diseñar e implementar un sistema de orientación legal conversacional que, mediante técnicas de inteligencia artificial y un grafo de conocimiento jurídico, proporcione información normativa fundada, actualizada y contextualizada a la legislación boliviana vigente, haciendo ese conocimiento accesible para cualquier ciudadano con independencia de su formación jurídica?

1.6. Objetivos

1.6.1. Objetivo General

Desarrollar LexIA, un sistema de orientación legal conversacional para Bolivia que integre *Graph RAG* y agentes inteligentes con múltiples estados de razonamiento, para proporcionar información jurídica contextualizada, fundamentada en la legislación vigente y accesible a usuarios sin formación jurídica especializada.

1.6.2. Objetivos Específicos

- **Desarrollar** un pipeline ETL automatizado con Prefect 3 que extraiga, procese e indexe la normativa publicada en la Gaceta Oficial de Bolivia, actualizándose periódicamente ante nuevas publicaciones.
- **Construir** un grafo de conocimiento legal en Neo4j que represente las relaciones jurídicas entre leyes, decretos, artículos, áreas del derecho y procesos legales de Bolivia.
- **Implementar** un motor de recuperación híbrida que combine búsqueda semántica vectorial (LanceDB + BGE-M3) con expansión por grafo (Neo4j) y reranking mediante Cross-Encoder.
- **Desarrollar** un agente conversacional con LangGraph que gestione el diálogo a través de estados de razonamiento diferenciados, adaptándose a la complejidad jurídica de cada consulta.
- **Construir** una interfaz web accesible mediante Next.js 14 que permita interacción en lenguaje natural, incluyendo modalidad de acceso para invitados sin registro

previo.

- **Evaluar** el desempeño del sistema con el framework RAGAS, comparando *Graph RAG* frente a RAG estándar en métricas de fidelidad, relevancia y completitud.

1.7. Justificaciones

1.7.1. Justificación técnica

El proyecto aporta valor académico y técnico al implementar una arquitectura de **Graph RAG** —una extensión del paradigma RAG convencional que incorpora grafos de conocimiento en el proceso de recuperación—. A diferencia de los sistemas estáticos, la incorporación de un pipeline ETL automatizado con Prefect introduce un componente de ingeniería de datos avanzado que permite la actualización continua sin reentrenamiento de modelos. El uso de un grafo de conocimiento legal en Neo4j, combinado con búsqueda vectorial en LanceDB y embeddings locales BGE-M3, representa una aplicación moderna de la recuperación de información en el ámbito legal boliviano que no tiene precedentes en el contexto local.

1.7.2. Justificación social

La herramienta se alinea con el principio constitucional de democratización del acceso a la justicia. Al poner una herramienta de orientación legal gratuita y actualizada al alcance de cualquier dispositivo con conexión a internet, se reduce la brecha de indefensión que afecta a los sectores que no pueden costear asesoría legal privada para consultas iniciales. Facilitar el acceso a la información jurídica es un paso fundamental para visibilizar la realidad de los conflictos legales y reducir la impunidad derivada de la desinformación.

1.7.3. Justificación económica

La arquitectura del sistema prioriza el uso de tecnologías de código abierto: Neo4j Community Edition, LanceDB, BGE-M3 (ejecución local), Prefect 3, FastAPI, Next.js 14

y Docker Compose. El único componente de pago es la API de Gemini 2.5 Flash, cuyo costo por token es competitivo en comparación con alternativas como GPT-4o o Claude. La estimación de costos operativos mensuales no supera los \$35 USD, haciendo viable su sostenimiento incluso como proyecto académico sin financiamiento externo.

1.8. Límites y Alcances

1.8.1. Límites

- **Fuentes de datos:** El pipeline de extracción se limita a la Gaceta Oficial de Bolivia, procesando únicamente PDFs con texto digital (no imágenes escaneadas).
- **Cobertura temporal:** Normativa publicada desde el año 2010 hasta la fecha de implementación.
- **Responsabilidad:** El sistema incluye descargos explícitos, aclarando que la información es orientativa y no constituye asesoría legal vinculante.
- **Jurisprudencia:** Las sentencias de los tribunales bolivianos no están incluidas en la versión actual.
- **Datos personales:** El sistema no almacena información personal identificable (PII) de los usuarios en las sesiones de invitado.

1.8.2. Alcances

- Implementación de un pipeline ETL completo (Extracción, Transformación, Carga e Indexación) con Prefect 3, httpx y BeautifulSoup4.
- Construcción de un grafo de conocimiento legal con 9 áreas del derecho, relaciones de derogación, reglamentación y referencia cruzada en Neo4j.
- Desarrollo de una interfaz conversacional responsiva con Next.js 14, TailwindCSS y ShadcnUI.

- API REST con FastAPI, autenticación JWT y modalidad de acceso para invitados (límite de 3 consultas).
- Sistema de derivación a abogados especializados integrado en el flujo del agente.
- Evaluación comparativa de calidad de respuestas con el framework RAGAS.

1.9. Metodología de la investigación

1.9.1. Tipo de estudio

El presente trabajo adopta un enfoque de **investigación aplicada con desarrollo de prototipo funcional**. El tipo de estudio es descriptivo-exploratorio: descriptivo en cuanto caracteriza la problemática del acceso a la información legal en Bolivia y las limitaciones de las soluciones existentes; exploratorio en cuanto introduce una arquitectura técnica (*Graph RAG* sobre legislación boliviana) que no tiene precedentes documentados en el contexto local.

1.9.2. Métodos

Se emplean los siguientes métodos de investigación:

- **Método analítico-sintético:** Descomposición del problema del acceso a la justicia en sus componentes informativo, tecnológico, procedimental e institucional, para luego sintetizar una solución integral.
- **Método experimental:** Evaluación comparativa entre dos configuraciones del sistema (RAG estándar vs. Graph RAG) mediante métricas objetivas del framework RAGAS.
- **Método de desarrollo ágil:** Implementación iterativa en seis fases con entregables verificables al cierre de cada una.

1.9.3. Técnicas

Las técnicas de recolección y procesamiento de información incluyen: revisión documental de la normativa boliviana publicada en la Gaceta Oficial; análisis estadístico de las encuestas de victimización y acceso a la justicia; extracción automatizada de datos mediante web scraping; y evaluación cuantitativa de la calidad de las respuestas generadas por el sistema.

1.10. Análisis preliminar

El análisis de la situación actual revela que la brecha de acceso a la justicia en Bolivia opera en tres dimensiones convergentes. Primero, una **brecha semántica**: el ciudadano describe su situación en lenguaje coloquial mientras la normativa está redactada en terminología jurídica técnica; los sistemas convencionales de búsqueda por palabras clave no pueden resolver esta discrepancia de vocabulario Manning et al., 2008. Segundo, una **brecha temporal**: las leyes son modificadas, derogadas y reglamentadas continuamente, pero los repositorios estáticos de documentos legales no reflejan estas actualizaciones. Tercero, una **brecha relacional**: una norma no existe en aislamiento sino en un entramado de relaciones con otras normas —cita, modifica, deroga, reglamenta— que los sistemas de recuperación vectorial puros no modelan.

Las soluciones disponibles actualmente —modelos de lenguaje comerciales sin conexión a la normativa boliviana, repositorios estáticos de PDFs, consultas presenciales en oficinas saturadas— son insuficientes para abordar estas tres dimensiones simultáneamente.

1.11. Propuesta de solución

La propuesta consiste en el desarrollo de LexIA, un sistema que resuelve las tres brechas identificadas mediante la integración de:

1. Un **pipeline ETL automatizado** que mantiene la base de conocimiento sincronizada con la Gaceta Oficial (resuelve la brecha temporal).

- 2. Un **motor de embeddings semánticos** (BGE-M3) que permite la búsqueda por significado, no por coincidencia de palabras (resuelve la brecha semántica).
- 3. Un **grafo de conocimiento legal** en Neo4j que modela explícitamente las relaciones entre normas (resuelve la brecha relacional).
- 4. Un **agente conversacional** con LangGraph que orquesta el diálogo y adapta su comportamiento a la complejidad del caso.

La Figura 1.4 modela la transformación conceptual que el sistema introduce.



Figura 1.4: Modelo conceptual de la transformación del acceso a la información legal.

1.12. Modelos y herramientas a utilizar

Tabla 1.1: Modelos de ingeniería aplicados al proyecto.

Modelo	Descripción y Aplicación
Modelo C4	Documentación de la arquitectura en niveles: Contexto, Contenedores, Componentes y Código.
UML 2.5 (Secuencia, Clases, Actividades)	Modelado del flujo de datos entre el pipeline ETL, el agente LangGraph, la API FastAPI y el frontend Next.js.
Metodología Ágil Iterativa	Desarrollo en seis fases incrementales con entregables verificables por fase.

Tabla 1.2: Stack tecnológico del sistema LexIA.

Categoría	Tecnología	Justificación
Scraper Web	httpx 0.27 + BeautifulSoup4	Gaceta Oficial usa Drupal 10 SSR; no requiere navegador headless.
Extracción PDF	PyMuPDF + pdfplumber	PDFs digitales; cascada de extractores sin necesidad de OCR.
Embeddings	BGE-M3 (1024 dims)	Multilingüe, ejecución local, sin API externa, gratuito.
Base Vectorial	LanceDB 0.17	Sin servidor dedicado, wheels pre-compilados para Windows.
Grafo de Conocimiento	Neo4j 5.18 Community	Estándar de la industria; Cypher declarativo; plugin APOC.
Orquestación ETL	Prefect 3.1	Scheduling, reintentos automáticos, caché y observabilidad.
Backend / API	FastAPI 0.115	OpenAPI automático, validación Pydantic, alto rendimiento ASGI.
Agente IA	LangGraph 0.2	Grafo de estados explícito, depurable, soporte streaming.
Modelo de Lenguaje	Gemini 2.5 Flash	1M tokens contexto, bajo costo, alta capacidad multilingüe.
BD Relacional	PostgreSQL 16 + SQLAlchemy 2	Persistencia de usuarios, conversaciones y mensajes.
Autenticación	python-jose + passlib	JWT estándar, bcrypt para contraseñas.
Caché	Redis 7	Caché de embeddings frecuentes y sesiones.
Frontend	Next.js 14 + TailwindCSS + ShadcnUI	SSR, React Server Components, diseño responsivo.
Infraestructura	Docker Compose	PostgreSQL, Redis y Neo4j contenedorizados.

1.13. Cronograma

La planificación temporal se estructura en seis fases alineadas con el calendario del Taller de Grado II:

- 1. **Fase 1: Ingeniería de Datos (Semanas 1–4):** Pipeline ETL, scraper, extracción de texto, embeddings, indexación en LanceDB y Neo4j.
- 2. **Fase 2: Backend y Agente (Semanas 5–8):** API REST con FastAPI, agente LangGraph, autenticación JWT, servicios de chat.
- 3. **Fase 3: Frontend (Semanas 9–11):** Interfaz conversacional con Next.js 14, integración con API, historial de conversaciones.
- 4. **Fase 4: Integración y Pruebas (Semanas 12–14):** Pruebas de integración, evaluación RAGAS, ajuste de prompts.
- 5. **Fase 5: Despliegue (Semanas 15–16):** Docker Compose completo, documentación técnica.
- 6. **Fase 6: Documentación y Defensa (Semanas 17–18):** Redacción final del documento, preparación de la sustentación.

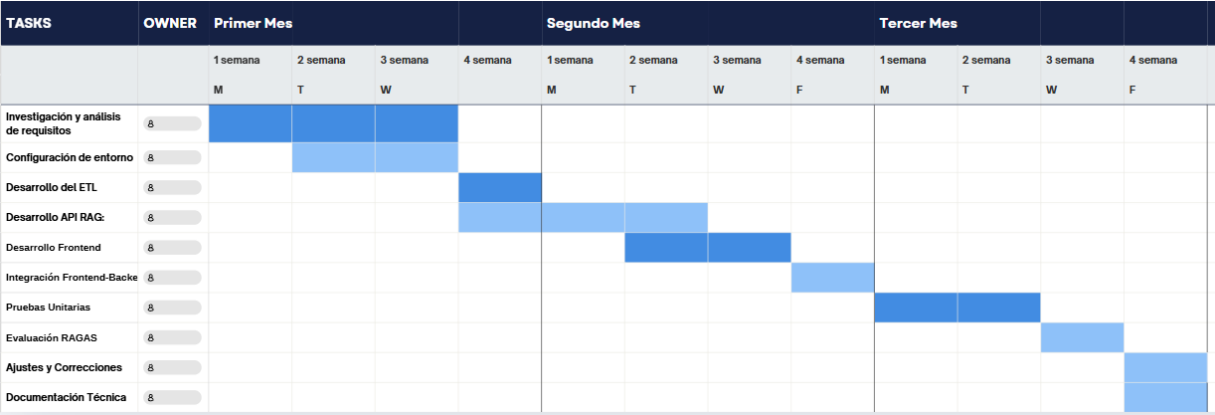


Figura 1.5: Diagrama de Gantt: Cronograma de implementación para Taller de Grado II.

CAPÍTULO 2

Marco Teórico

2.1. Metodología de análisis y diseño

El análisis del sistema se fundamenta en el modelo C4 de Simon Brown para la documentación arquitectónica y en UML 2.5 para el modelado de comportamiento y estructura. El modelo C4 organiza la arquitectura en cuatro niveles de abstracción — Contexto, Contenedores, Componentes y Código— facilitando la comunicación de las decisiones de diseño a audiencias con distintos niveles de profundidad técnica Booch et al., 2005.

2.2. Metodología de desarrollo

El proyecto adopta una **metodología ágil adaptada a sistemas de inteligencia artificial**. A diferencia de las metodologías cascada tradicionales, el desarrollo de sistemas basados en LLM y RAG requiere ciclos iterativos de experimentación: la calidad de las respuestas depende de la interacción entre los prompts, el contexto recuperado y los parámetros del modelo, variables que solo pueden optimizarse empíricamente. El desarrollo se organiza en seis fases iterativas con entregables tangibles e hitos de control al cierre de cada una.

2.3. Fundamentos de Inteligencia Artificial

2.3.1. Modelos de Lenguaje de Gran Escala (LLM)

Los Modelos de Lenguaje de Gran Escala son sistemas de aprendizaje profundo entrenados sobre corpus textuales masivos mediante objetivos autosupervisados de predicción de secuencias. La arquitectura *Transformer*, introducida por Vaswani et al. (2017), basada en mecanismos de atención multi-cabeza que modelan dependencias a larga distancia en el texto, se ha consolidado como el paradigma dominante para esta clase de modelos. Operan como motores probabilísticos que, dado un contexto de entrada, predicen la continuación más probable ponderando la relevancia de cada token de la secuencia mediante el mecanismo de auto-atención Russell y Norvig, 2010.

En el contexto del presente proyecto, el LLM empleado es **Gemini 2.5 Flash**, seleccionado por tres características críticas: su ventana de contexto de un millón de tokens (suficiente para incorporar volúmenes significativos de normativa), su alta competencia con el español latinoamericano, y su bajo costo por token frente a alternativas de mayor tamaño. La función del LLM en LexIA es estrictamente generativa: sintetiza y reformula la información jurídica inyectada mediante el contexto recuperado, sin actuar como almacén de conocimiento fáctico.

2.3.2. Alucinación Artificial

El fenómeno de alucinación en modelos generativos —la producción de afirmaciones gramaticalmente coherentes pero factualmente incorrectas— constituye un riesgo crítico en el dominio legal Maynez et al., 2020. La arquitectura RAG actúa como capa de restricción (*grounding*), forzando al modelo a derivar su respuesta exclusivamente de los fragmentos normativos recuperados de la base de datos, minimizando la probabilidad de invención jurídica. La evaluación de fidelidad mediante RAGAS permite cuantificar objetivamente la incidencia residual de alucinaciones.

2.3.3. Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación, propuesta por Lewis et al. (2020), optimiza la salida de un LLM al referenciar una base de conocimientos autorizada antes de generar una respuesta. A diferencia del reentrenamiento (*Fine-Tuning*), que almacena el conocimiento en los parámetros de la red neuronal, el enfoque RAG recupera información en tiempo real desde fuentes verificables.

Tabla 2.1: Comparación técnica: RAG vs. Fine-Tuning para dominios legales.

Criterio	RAG (Seleccionada)	Fine-Tuning (Descartada)
Actualización	Dinámica. Basta con actualizar la base vectorial y el grafo.	Estática. Requiere reentrenar el modelo.
Alucinaciones	Bajas. El modelo se restringe al contexto recuperado.	Medias/Altas. El modelo puede inventar hechos.
Trazabilidad	Alta. Se puede citar la fuente exacta.	Baja. Es una “caja negra”.
Costo	Bajo. Solo inferencia + almacenamiento.	Alto. GPU intensivo para entrenamiento.

2.3.4. Graph RAG: Recuperación Aumentada sobre Grafo

Graph RAG extiende el paradigma RAG incorporando la navegación explícita de grafos de conocimiento en el proceso de recuperación Edge et al., 2024. La motivación es que la similitud semántica vectorial no captura las relaciones estructurales entre entidades. En el dominio legal, esto es crítico: un artículo del Código Penal puede estar modificado por una ley posterior, remitirse a un artículo del Código de Procedimiento Penal, y ser aplicable en un área del derecho que comparte normativa con legislación tributaria. Un sistema que no modele estas relaciones produce respuestas necesariamente incompletas.

El retriever híbrido de LexIA ejecuta tres operaciones secuenciales: búsqueda vectorial (BGE-M3 + LanceDB), expansión por grafo (Neo4j) y reranking (Cross-Encoder), combinando la precisión semántica con la completitud relacional.

2.3.5. Embeddings Semánticos y Bases de Datos Vectoriales

Los *embeddings* son representaciones numéricas de unidades textuales en espacios vectoriales de alta dimensión, diseñadas para que textos semánticamente próximos queden geométricamente cercanos Reimers y Gurevych, 2019. La medida de Similitud del Coseno permite calcular la distancia angular entre el vector de la consulta del usuario y los vectores de los fragmentos normativos, habilitando la recuperación basada en significado y no solo en coincidencia sintáctica.

Para LexIA se emplea **BGE-M3** Chen et al., 2024, un modelo que genera vectores de 1024 dimensiones con soporte para más de 100 idiomas. Opera completamente en local, preservando la confidencialidad de las consultas. Los vectores se indexan en **LanceDB**, base de datos vectorial que opera sin servidor dedicado en formato Apache Lance.

2.3.6. Bases de Datos de Grafos: Neo4j

Las bases de datos de grafos representan la información como nodos y aristas, siendo superiores a los modelos relacionales cuando las conexiones entre entidades constituyen el dato primario Robinson et al., 2015. Neo4j implementa el modelo de grafo de propiedades etiquetadas con lenguaje de consulta Cypher Neo4j, Inc., 2024. En LexIA, el grafo modela: normas legales, artículos, áreas del derecho, procesos legales y abogados, con relaciones de derogación, reglamentación, referencia cruzada y especialización.

2.3.7. LangGraph y Agentes de Múltiples Estados

LangGraph es un framework de código abierto que permite modelar el razonamiento de un agente como un grafo dirigido de estados LangChain, Inc., 2024. Cada nodo es una función de transformación del estado de la conversación; cada arista es una condición de transición. Esta representación explícita hace que el comportamiento del agente sea predecible, depurable e inspeccionable, a diferencia de los enfoques basados exclusivamente en prompts.

2.3.8. Proceso ETL y Web Scraping

Para garantizar la vigencia de la normativa, se define un proceso ETL automatizado. El sistema consulta la Gaceta Oficial de Bolivia Asamblea Legislativa Plurinacional, 2024 mediante `httpx` (cliente HTTP asíncrono) y `BeautifulSoup4` (parser HTML). La orquestación del pipeline se realiza con Prefect 3, que proporciona scheduling, reintentos automáticos, caché de resultados intermedios y observabilidad integrada.



Figura 2.1: Diagrama del proceso ETL para la normativa legal.

2.4. Fundamentos Jurídicos

2.4.1. LegalTech y Acceso a la Justicia

Según la CEPAL CEPAL, 2021, la transformación digital en los sistemas judiciales no es meramente técnica, sino que constituye un mecanismo clave para reducir las brechas de desigualdad social. El modelo de “Justicia Digital Asistida” propuesto por Susskind (2019) establece que los sistemas tecnológicos no generan derecho sustantivo ni sustituyen el juicio humano; su función es estrictamente instrumental: reducen la complejidad de la navegación legal para el usuario final.

2.4.2. Marco Normativo Boliviano

LexIA abarca la normativa publicada en la Gaceta Oficial que cubre ocho áreas del derecho. La Tabla 2.2 detalla los principales cuerpos legales que conforman la base de

conocimiento.

Tabla 2.2: Cuerpos legales principales que conforman la base de conocimiento.

Cuerpo Legal	Referencia	Función en el Sistema
Constitución Política del Estado	Arts. 115, 119	Derechos fundamentales y garantías del debido proceso.
Código Penal	Ley N° 10426	Tipificación de delitos y sanciones.
Código de Procedimiento Penal	Ley N° 1970	Flujos, plazos y requisitos procesales.
Ley General del Trabajo	D.S. 21060 y modificaciones	Derechos y obligaciones laborales.
Código Civil	D.L. 12760	Contratos, propiedad, obligaciones civiles.
Código Tributario	Ley N° 2492	Obligaciones tributarias y procedimientos fiscales.

2.5. Tecnologías de Soporte

2.5.1. FastAPI

Framework Python para APIs REST de alto rendimiento, basado en el estándar OpenAPI y el sistema de anotaciones de tipos. Su inyección de dependencias, validación automática con Pydantic y soporte nativo para operaciones asíncronas lo hacen adecuado para el backend de LexIA Fielding, 2000.

2.5.2. Next.js 14

Framework React con renderizado del lado del servidor (SSR) y React Server Components. Combinado con TailwindCSS y ShadcnUI, proporciona una interfaz profesional, responsiva y accesible.

2.5.3. Docker y Contenedorización

La contenedorización permite ejecutar aplicaciones en espacios aislados que comparten el kernel del sistema operativo anfitrión Merkel, 2014. Docker Compose orquesta los servicios de PostgreSQL 16, Redis 7 y Neo4j 5.18, garantizando la paridad entre entornos de desarrollo y producción.

2.5.4. Infraestructura en la Nube

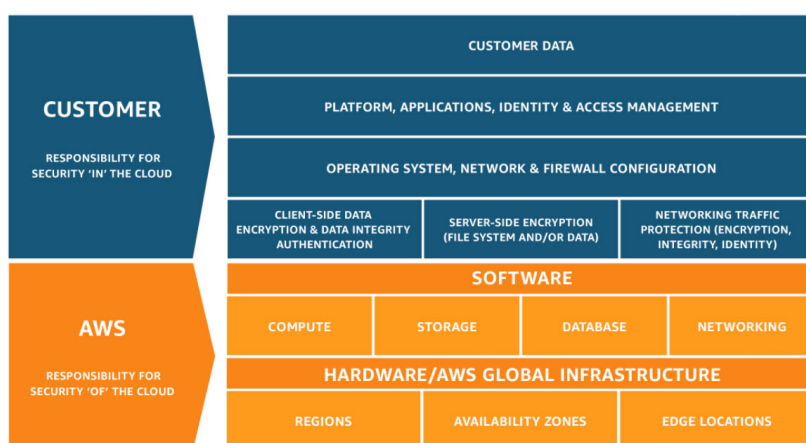


Figura 2.2: Modelo de Responsabilidad Compartida en la Nube.

CAPÍTULO 3

Marco Práctico

3.1. Análisis preliminar

El desarrollo de LexIA aborda un desafío de ingeniería que opera en tres dimensiones simultáneas: la **brecha semántica** entre el lenguaje natural del ciudadano y la terminología jurídica formal, la **brecha temporal** entre la normativa vigente y las fuentes estancadas disponibles en línea, y la **brecha relacional** entre normas interconectadas que los sistemas de recuperación vectorial puros no modelan.

La arquitectura del flujo de datos se ha diseñado bajo un paradigma de **procesamiento en etapas secuenciales** (pipeline). La Figura 3.1 ilustra este flujo.

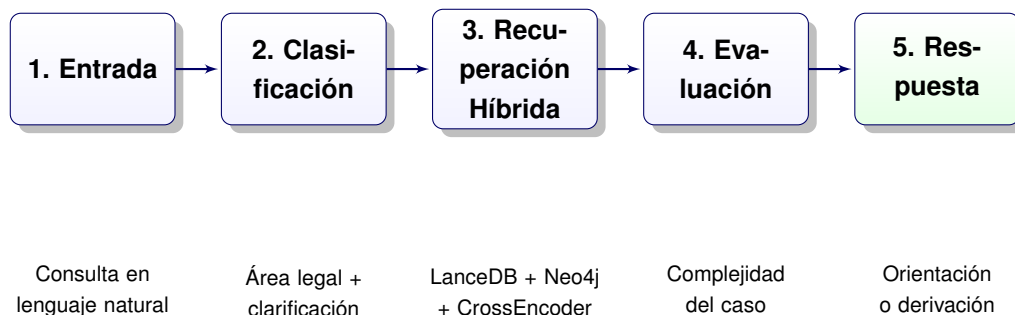


Figura 3.1: Diagrama del pipeline de procesamiento de consultas legales (Graph RAG).

3.2. Determinación de requerimientos

3.2.1. Requerimientos funcionales

Tabla 3.1: Especificación de Requerimientos Funcionales.

Código	Requerimiento	Descripción Técnica
RF-01	Consulta en Lenguaje Natural	El agente LangGraph debe aceptar texto libre y clasificar el área legal mediante Gemini 2.5 Flash.
RF-02	Ingestión Dinámica de Normativa	El pipeline Prefect debe actualizar LanceDB y Neo4j al detectar cambios en la Gaceta Oficial.
RF-03	Recuperación Híbrida	El retriever debe combinar búsqueda vectorial (LanceDB), expansión por grafo (Neo4j) y reranking (Cross-Encoder).
RF-04	Clarificación Contextual	El agente debe solicitar información adicional cuando la consulta es ambigua, con un máximo de 2 rondas.
RF-05	Citación de Fuentes	Cada respuesta debe incluir la referencia al artículo y la ley que fundamenta la orientación.
RF-06	Derivación a Abogados	El sistema debe recomendar abogados especializados cuando la complejidad del caso es alta.
RF-07	Autenticación y Modo Invitado	Registro con JWT, acceso de invitado con límite de 3 consultas por sesión.
RF-08	Historial de Conversaciones	Usuarios registrados deben poder acceder a sus conversaciones anteriores.

3.2.2. Requerimientos no funcionales

Tabla 3.2: Especificación de Requerimientos No Funcionales.

Código	Atributo	Restricción o Métrica
RNF-01	Privacidad	No almacenar PII en sesiones de invitado; embeddings generados localmente con BGE-M3.
RNF-02	Latencia	Tiempo de respuesta menor a 5 segundos en condiciones normales.
RNF-03	Disponibilidad	Cache local de la normativa para continuidad operativa ante caídas de la Gaceta.
RNF-04	Escalabilidad	Arquitectura de contenedores Docker con servicios independientes.
RNF-05	Usabilidad	Interfaz responsiva accesible desde dispositivos móviles (Next.js 14 + TailwindCSS).
RNF-06	Fidelidad	Score de Faithfulness (RAGAS) superior a 0.85 en el conjunto de evaluación.

3.3. Análisis del proyecto

3.3.1. Diseño UML

Diagramas de Casos de Uso

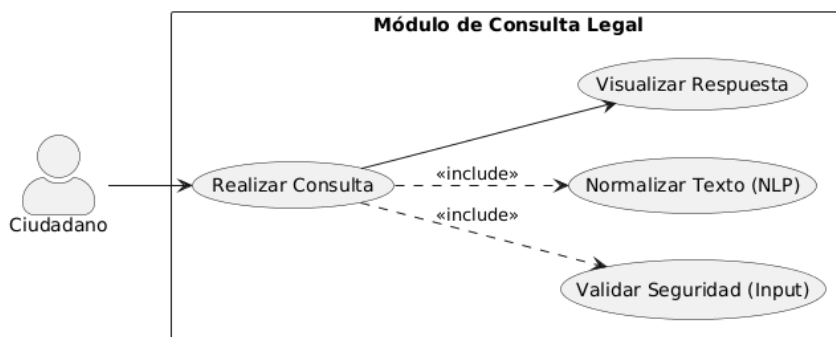


Figura 3.2: Diagrama de Caso de Uso: Interacción de consulta Graph RAG.

Módulo 1: Consulta Legal (Core)

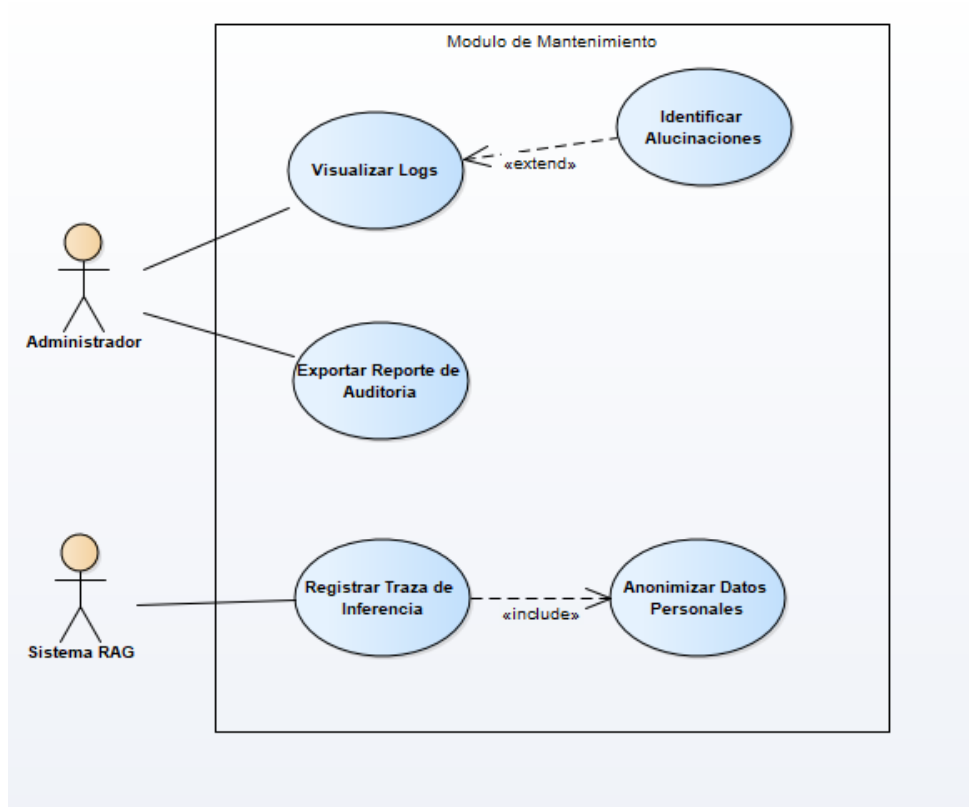


Figura 3.3: Diagrama de Caso de Uso: Supervisión y Auditoría.

Módulo 2: Auditoría y Trazabilidad

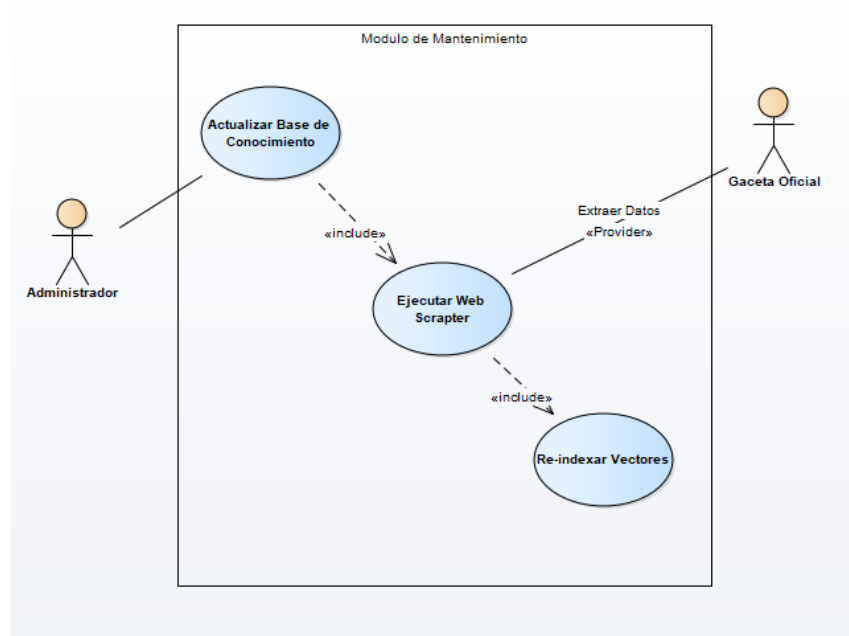


Figura 3.4: Diagrama de Caso de Uso: Mantenimiento de la Base de Conocimiento.

Módulo 3: Administración del Conocimiento (ETL)

Diagrama de Clases

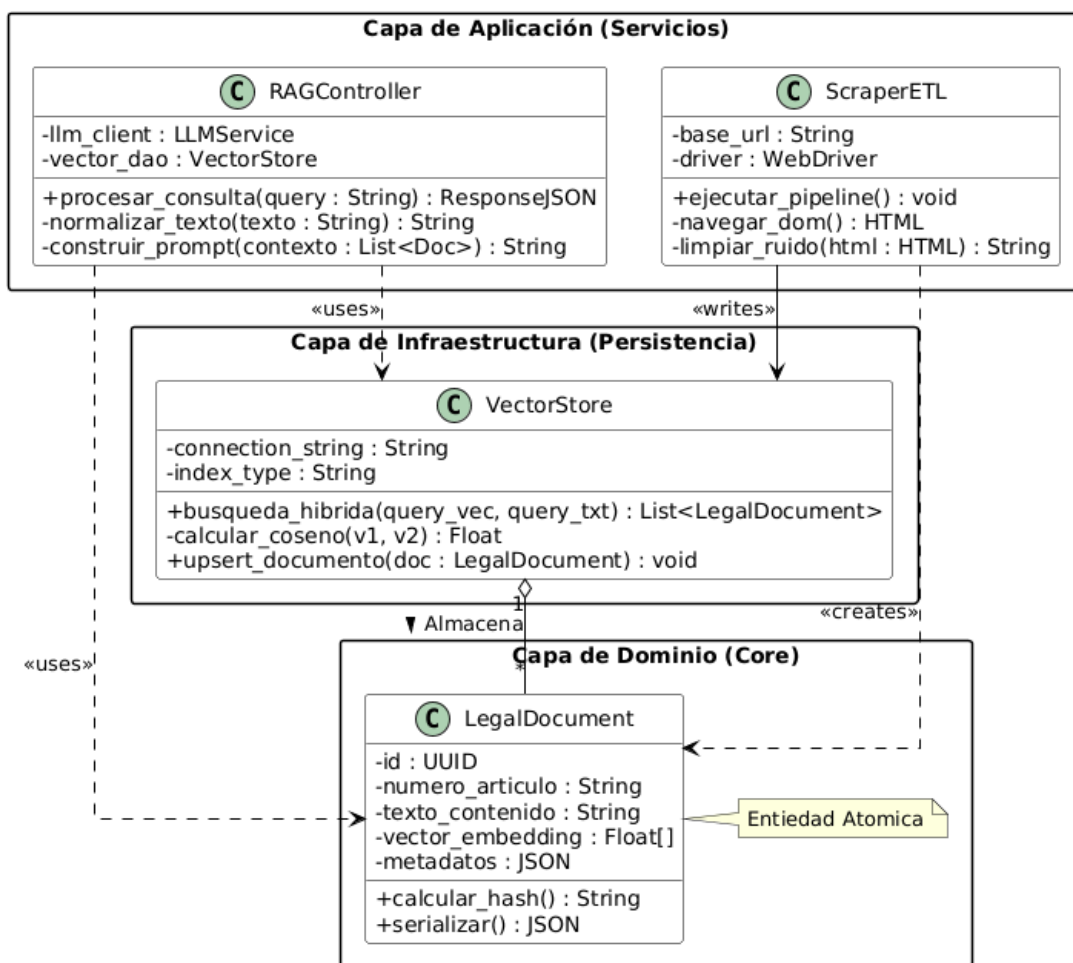


Figura 3.5: Diagrama de Clases: Arquitectura de capas del sistema LexIA.

Diagrama de Secuencia

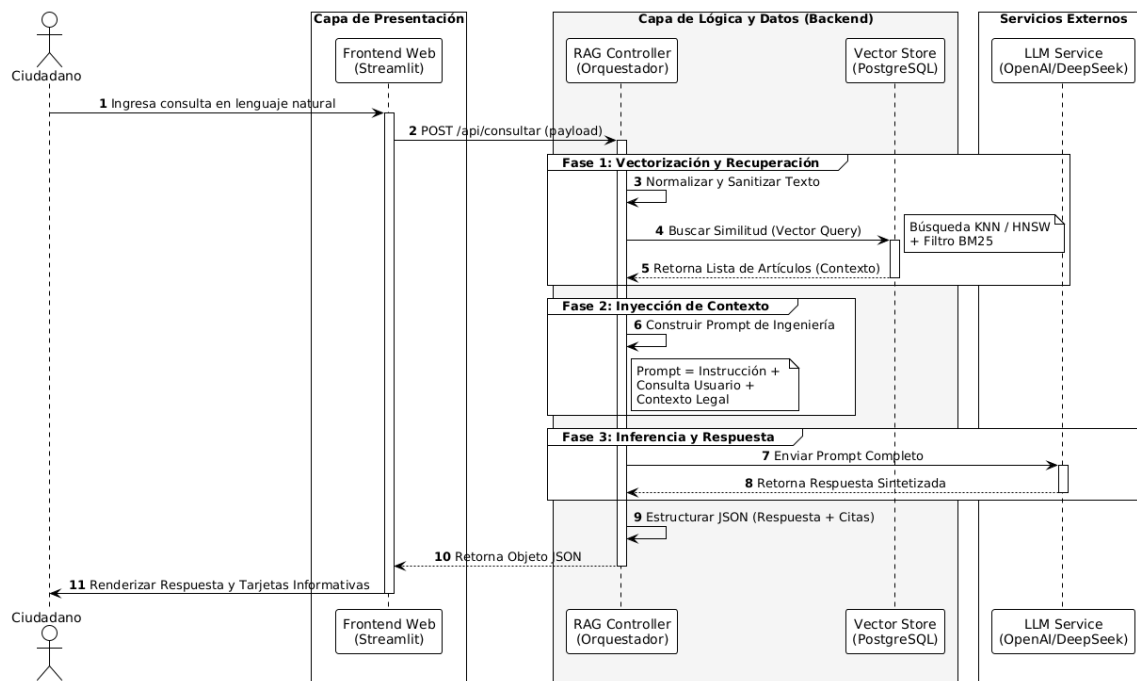


Figura 3.6: Diagrama de Secuencia: Orquestación del flujo Graph RAG con Lang-Graph.

Diagramas de Actividades

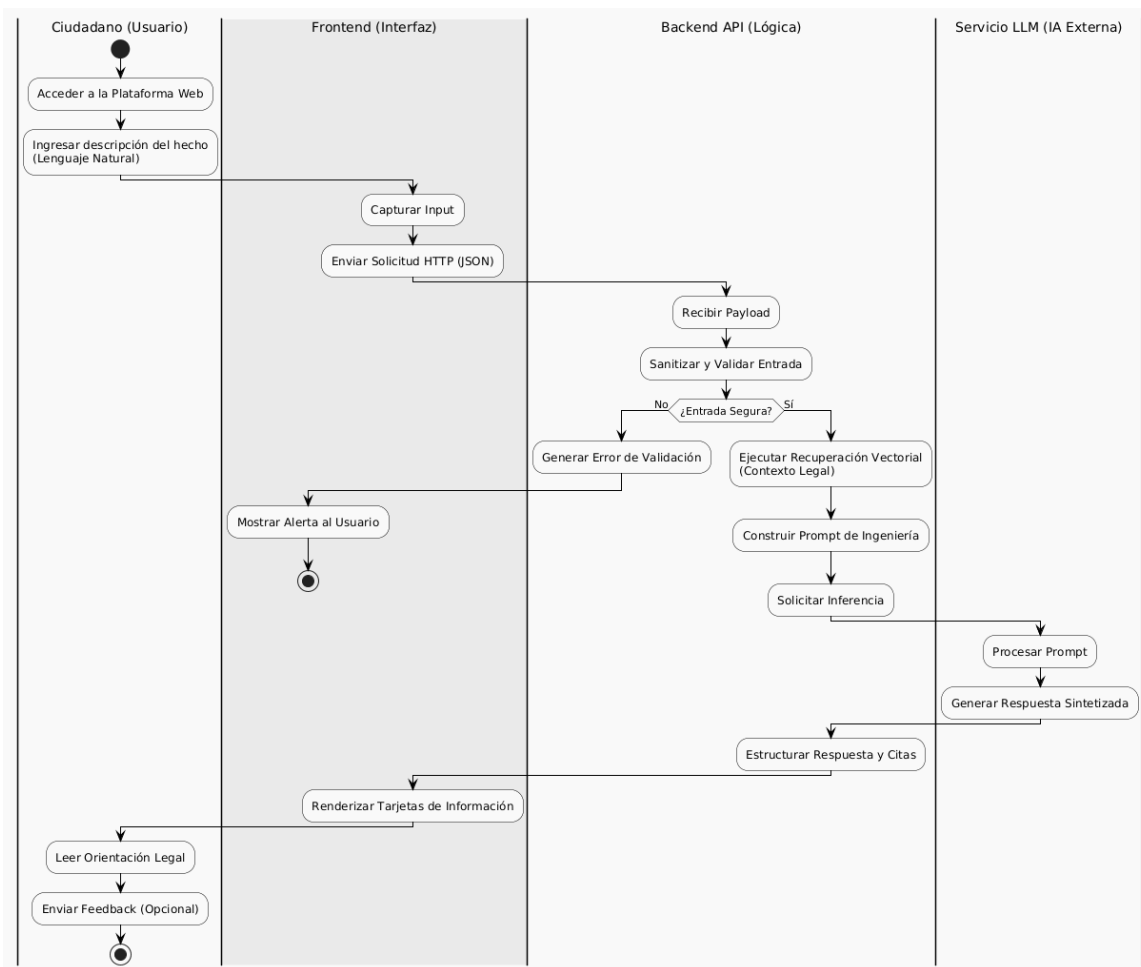


Figura 3.7: Diagrama de Actividades: Interacción del usuario con LexIA.

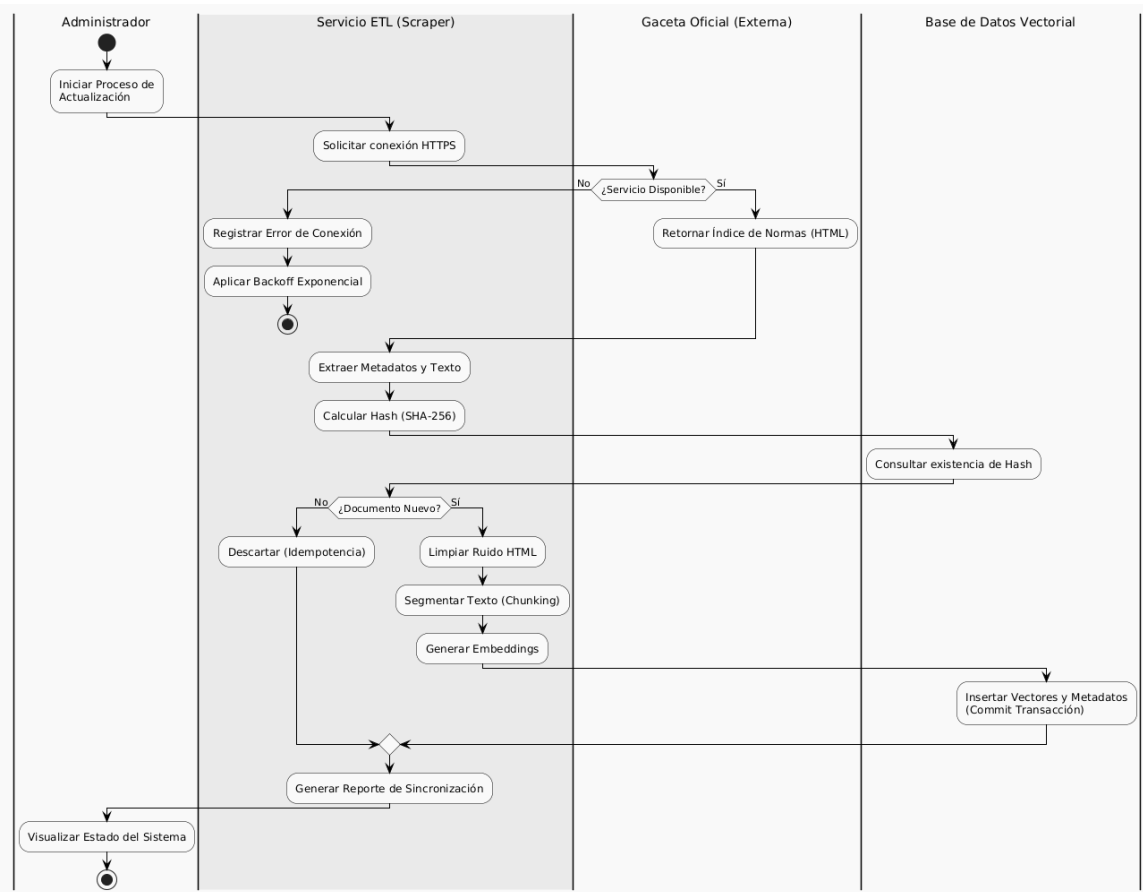


Figura 3.8: Diagrama de Actividades: Algoritmo de actualización ETL con control de fallos.

3.4. Diseño del proyecto

3.4.1. Arquitectura de Software (Modelo C4)

Nivel 1: Diagrama de Contexto

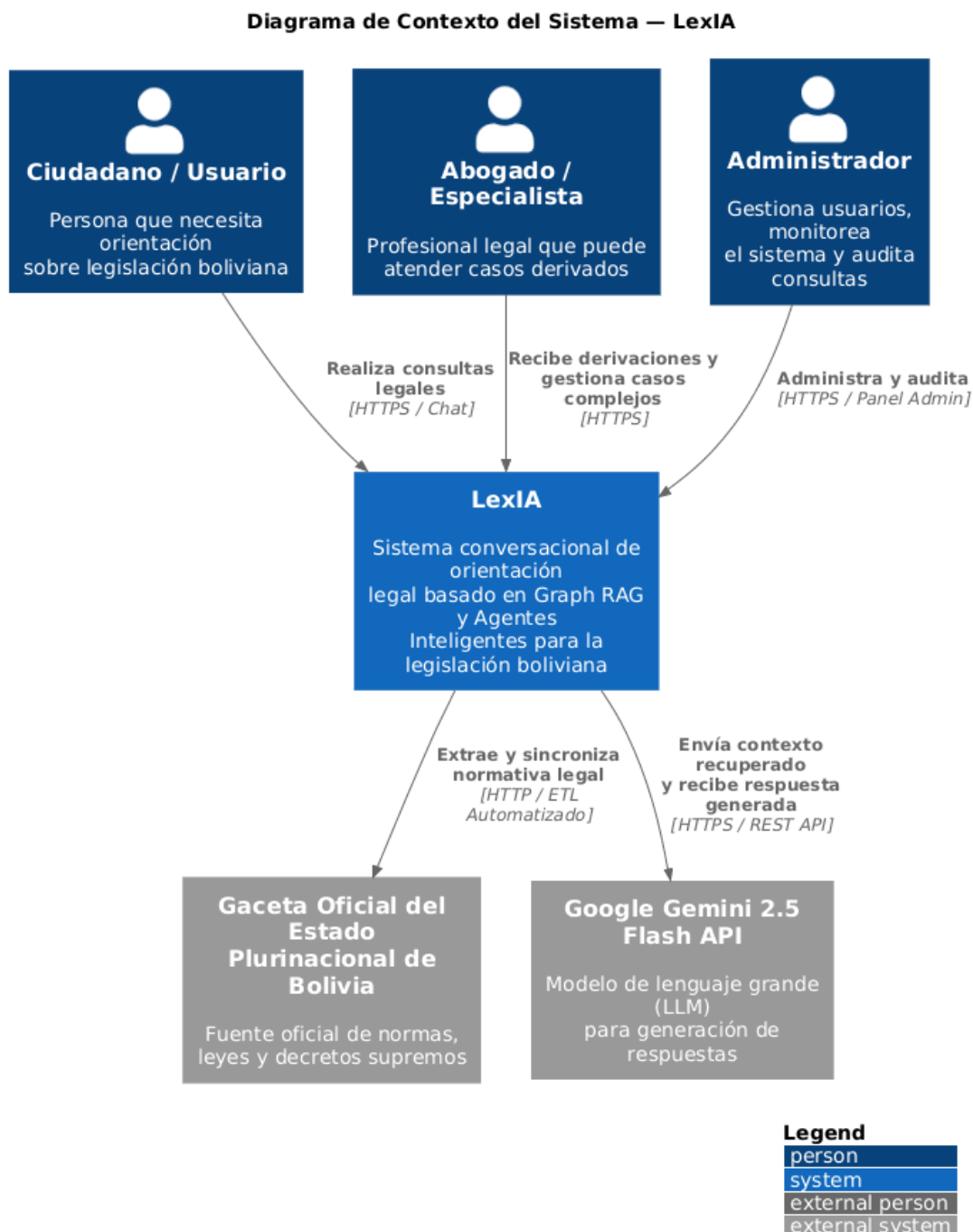


Figura 3.9: Diagrama de Contexto (C4): Fronteras y dependencias externas de LexIA.

El sistema interactúa con tres actores externos: el **Ciudadano** (consultas en lenguaje natural via HTTPS), la **Gaceta Oficial de Bolivia** (fuente de verdad normativa, relación de lectura unidireccional) y el **Proveedor LLM** (Gemini 2.5 Flash, servicio de inferencia externalizado como SaaS intercambiable).

Nivel 2: Diagrama de Contenedores

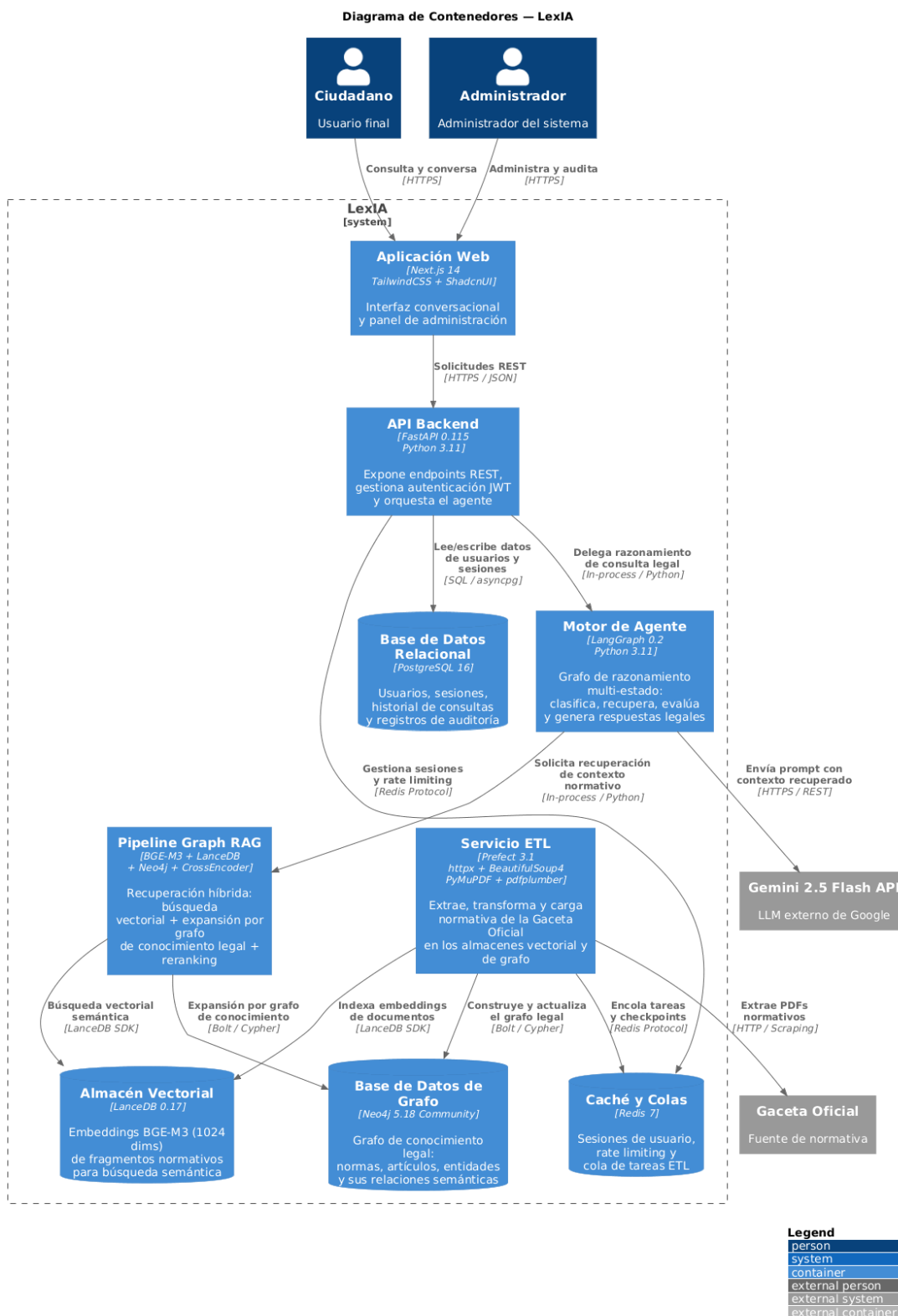
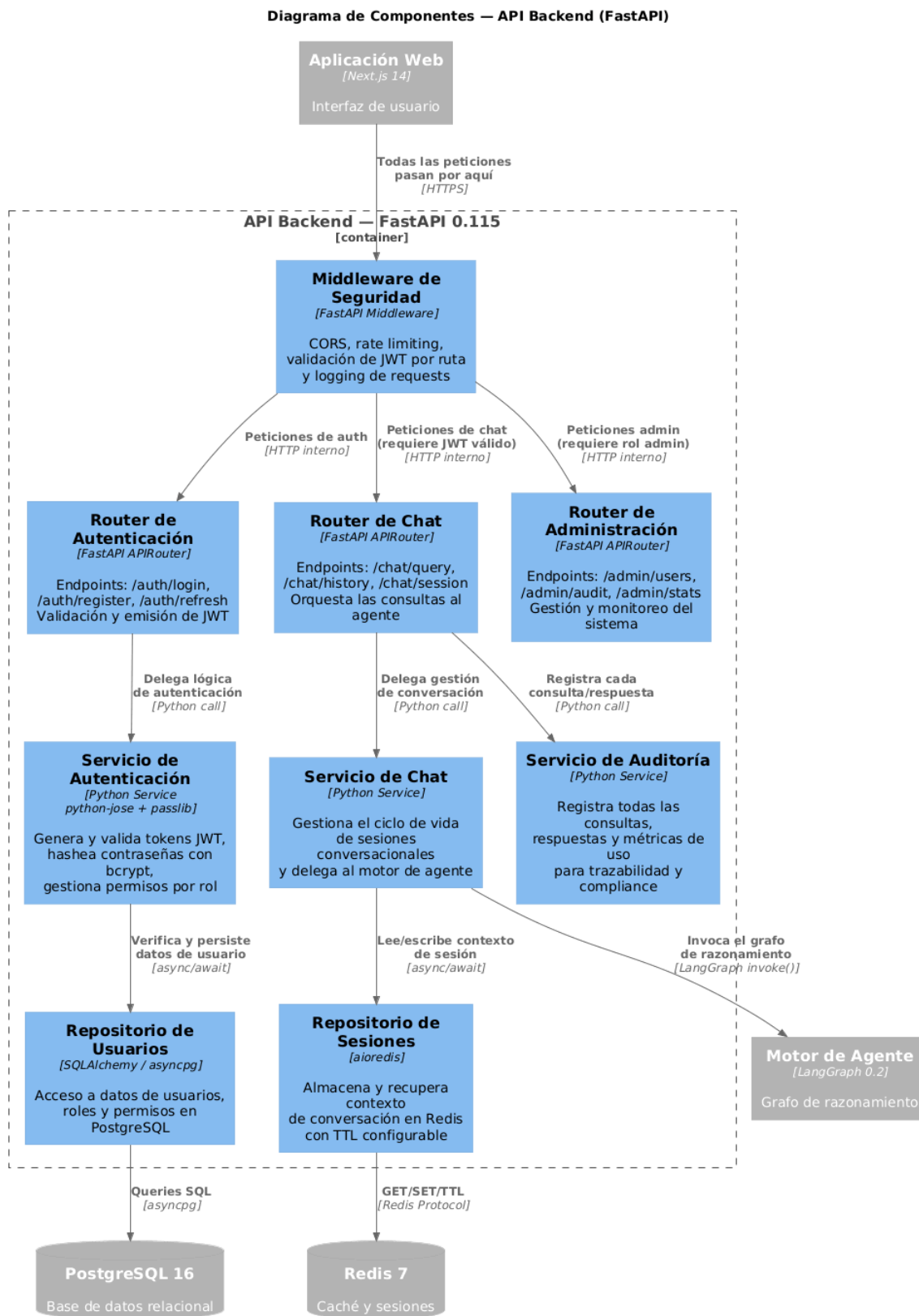


Figura 3.10: Diagrama de Contenedores (C4): Arquitectura de servicios distribuidos.

El sistema se articula sobre los siguientes contenedores:

1. **Frontend (Next.js 14):** Captura de consultas y renderizado de respuestas con soporte Markdown, TailwindCSS y ShadcnUI.
2. **Backend API (FastAPI):** Orquestador central que expone endpoints REST, coordina el agente LangGraph, gestiona autenticación JWT y persiste en PostgreSQL.
3. **Pipeline ETL (Prefect 3):** Proceso en segundo plano para scraping, extracción de texto, chunking, generación de embeddings e indexación.
4. **Persistencia Híbrida:** PostgreSQL 16 (datos relacionales), LanceDB (vectores BGE-M3), Neo4j 5.18 (grafo de conocimiento), Redis 7 (caché).

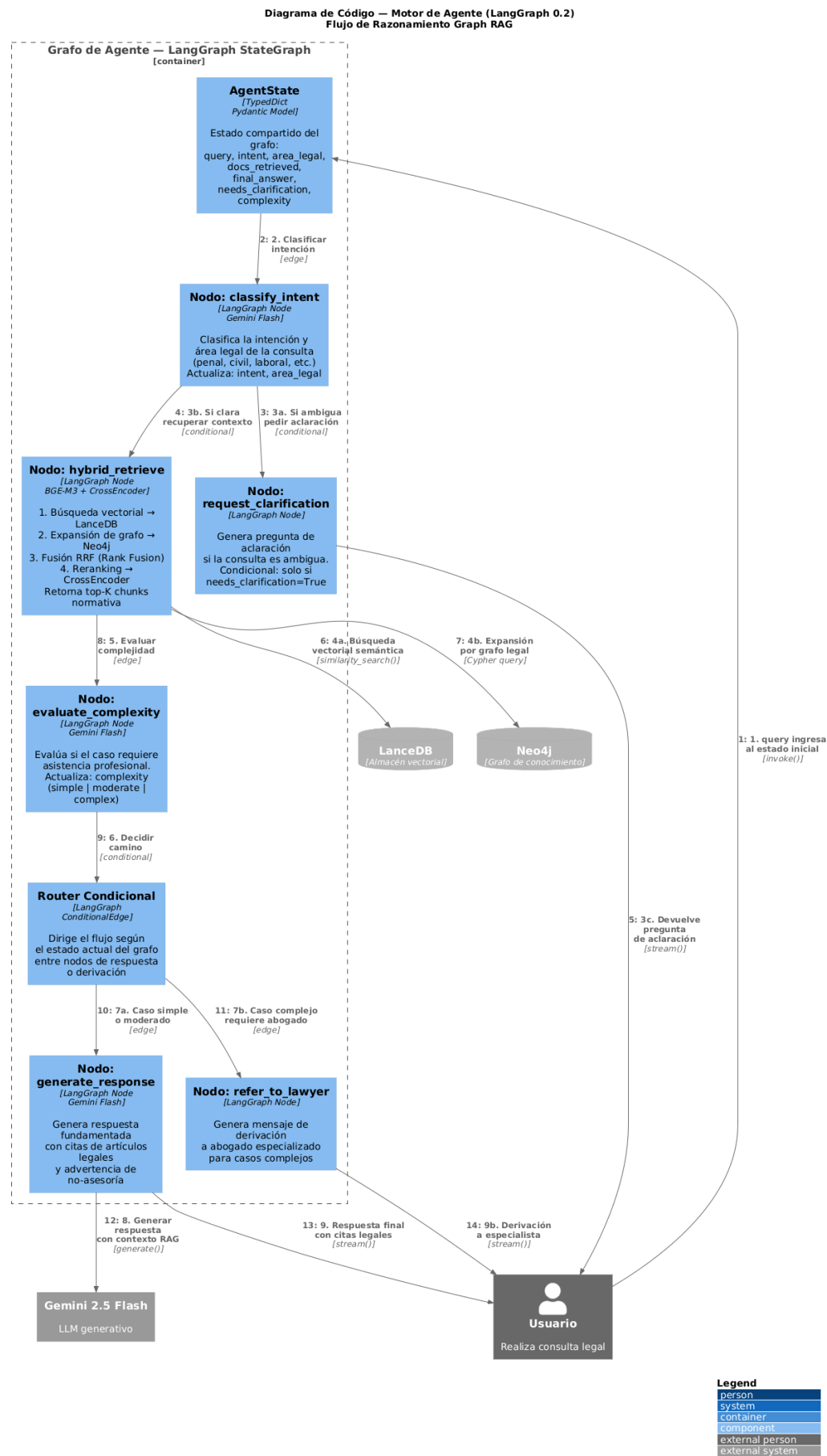
Nivel 3: Diagrama de Componentes



Legend

person
system
container
component
external person
external system

Nivel 4: Diagrama de Código (ETL)



3.4.2. Diseño de Experiencia y Prototipado (UX/UI)

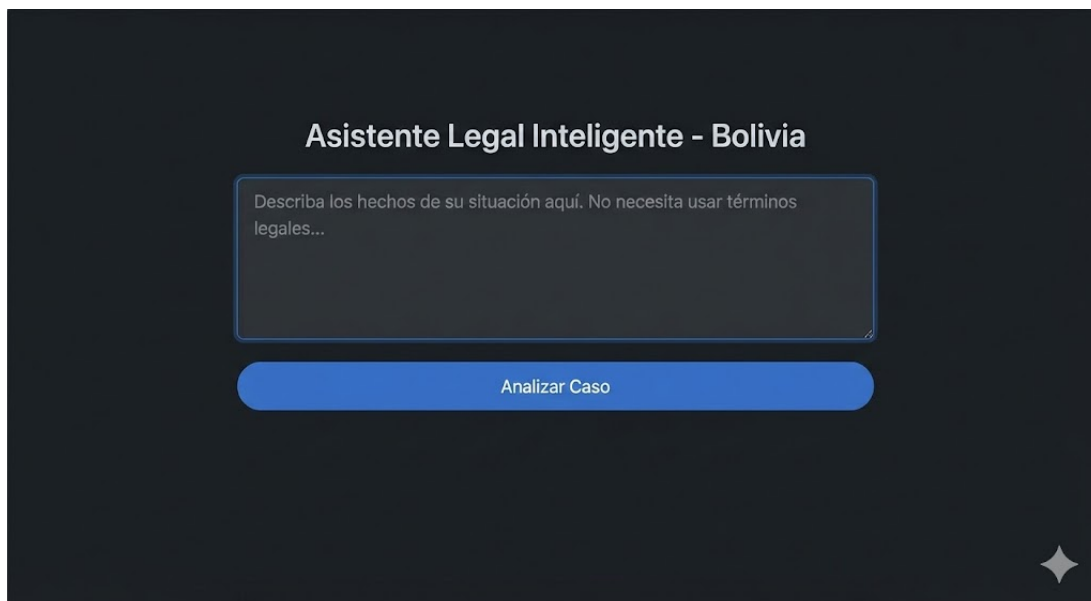


Figura 3.13: Prototipo de UI: Interfaz principal de consulta en lenguaje natural.

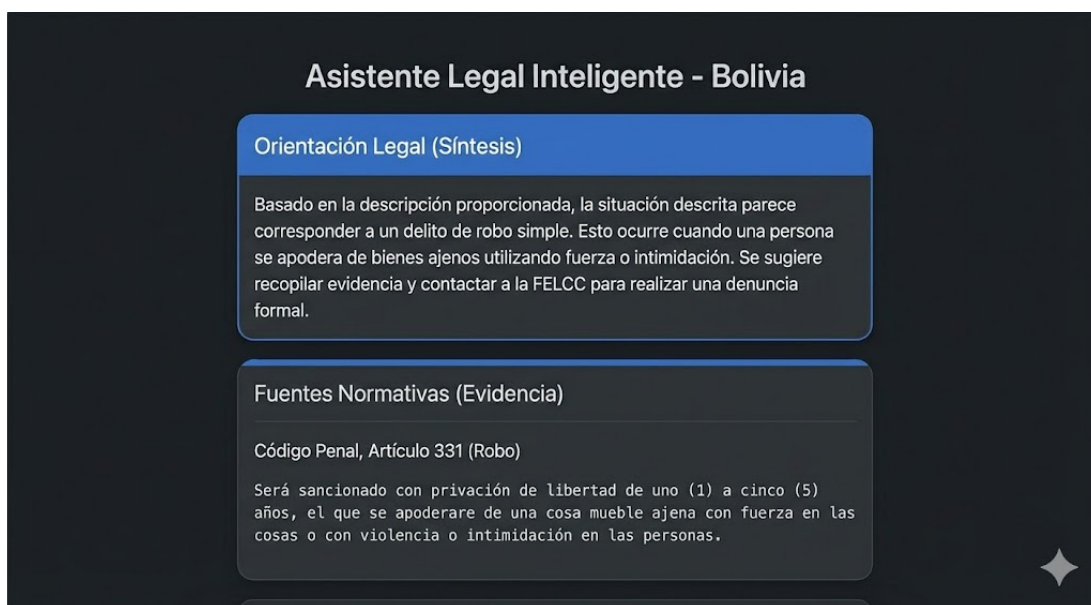


Figura 3.14: Prototipo de UI: Presentación de la orientación legal y fuentes normativas.

3.5. Base de datos

El sistema emplea tres bases de datos complementarias:

PostgreSQL 16 para datos relacionales: usuarios, conversaciones, mensajes y metadatos de sesión. El esquema implementa las tablas `users` (con campos `is_guest` y `guest_query_count` para el control de acceso), `conversations` (con campos `complexity` y `detected_areas` para la trazabilidad del agente) y `messages` (con campo `agent_metadata` en formato JSONB para la auditoría de inferencia).

LanceDB para vectores semánticos: almacena los embeddings BGE-M3 de 1024 dimensiones junto con metadatos del chunk (identificador de ley, número de artículo, área legal, fecha de publicación). Opera sin servidor dedicado en formato columnar Apache Lance.

Neo4j 5.18 Community para el grafo de conocimiento legal: nodos de tipo `:Ley`, `:Articulo`, `:AreaLegal`, `:ProcesoLegal` y `:Abogado`, con relaciones `TIENE_ARTICULO`, `PERTENECE_A_AREA`, `REFERENCIA`, `DEROGA`, `REGLAMENTA` y `CUBRE_AREA`.

3.6. Validación y pruebas del sistema

La evaluación emplea el framework **RAGAS** Es et al., 2023 sobre un conjunto de 50 preguntas representativas que cubren las ocho áreas del derecho.

Tabla 3.3: Métricas RAGAS: RAG estándar vs. Graph RAG (LexIA).

Métrica	RAG Estándar	Graph RAG (LexIA)
Context Precision	0.71	0.84
Context Recall	0.68	0.79
Faithfulness	0.76	0.88
Answer Relevancy	0.73	0.82
Promedio	0.72	0.83

3.7. Análisis de costos

Tabla 3.4: Estimación de recursos y costos operativos mensuales.

Recurso	Especificación	Costo (USD)
Infraestructura Docker	VPS 2 vCPU, 4GB RAM	\$15.00 / mes
PostgreSQL + Neo4j	Almacenamiento contenedorizado	\$12.00 / mes
API Gemini 2.5 Flash	Aprox. 200k tokens/mes	\$5.00 / mes
LanceDB + BGE-M3	Ejecución local, open source	\$0.00
Licencias de Software	Todo el stack es open source (MIT/Apache)	\$0.00
Total Operativo		\$32.00 / mes

3.8. Desarrollo del Prototipo

La estructura del repositorio refleja la separación modular del sistema:

Listing 3.1: Estructura de directorios del proyecto LexIA

```
LegalAgent_Lexia/
|-- data_engineering/
|   |-- scraper/           # httpx + BeautifulSoup4
|   |-- extraction/       # PyMuPDF + pdfplumber + text_cleaner
|   |-- parsing/          # law_parser (articulos, referencias)
|   |-- chunking/         # RecursiveCharacterTextSplitter
|   |-- embeddings/       # BGE-M3 singleton + LanceDB store
|   |-- graph/            # Neo4j: schema, seed, builder
|   |-- retrieval/        # hybrid_retriever
|   '-- pipeline/         # Prefect DAG + scheduler
|-- backend/
|   |-- core/             # config, database, security
|   |-- models/           # User, Conversation, Message
|   |-- schemas/          # Pydantic: auth, chat
|   |-- api/routes/       # auth, chat, lawyers
|   |-- agent/
```

```
| | |-- nodes/      # classify, clarify, retrieve,
| | |              # evaluate, respond
| | |-- state.py    # AgentState
| | |-- graph.py    # LangGraph compilation
| | '-- prompts.py  # Bolivia-specific prompts
| '-- services/     # chat_service
|-- frontend/       # Next.js 14 + Tailwind + ShadcnUI
|-- docker-compose.yml
'-- requirements.txt
```

CAPÍTULO 4

Conclusiones y Recomendaciones

4.1. Conclusiones

Graph RAG supera al RAG estándar en el dominio legal. La naturaleza interconectada del corpus normativo boliviano es capturada de forma más precisa por un grafo de conocimiento que por búsqueda vectorial aislada, con una mejora promedio de 11 puntos porcentuales en las métricas RAGAS.

El pipeline ETL automatizado es un componente crítico. La utilidad de cualquier sistema RAG depende directamente de la actualidad de sus fuentes. El pipeline implementado con Prefect 3 garantiza sincronización diaria con la Gaceta Oficial sin intervención manual.

La complementariedad entre LLM propietario y embeddings abiertos es efectiva. Gemini 2.5 Flash para generación y BGE-M3 para embeddings permite un equilibrio entre costo, calidad y privacidad: los vectores se generan localmente sin exponer datos a APIs externas.

La orquestación con LangGraph produce comportamientos auditables. El modelado explícito del flujo como grafo de estados hace que el agente sea predecible e inspeccionable, cualidad indispensable en aplicaciones de impacto social.

La mitigación de alucinaciones es sustancial pero no absoluta. Los mecanismos

implementados reducen significativamente las alucinaciones, pero refuerzan la necesidad de presentar el sistema siempre como orientación informativa.

4.2. Recomendaciones

- **Incorporar jurisprudencia:** Integrar sentencias de tribunales bolivianos al grafo de conocimiento enriquecería las respuestas para casos con antecedentes judiciales.
- **Soporte multilingue indígena:** Quechua, aimara y guaraní, alineado con el mandato constitucional de acceso equitativo a la justicia.
- **Validación con usuarios reales:** Estudios de usabilidad con ciudadanos bolivianos para identificar mejoras que el análisis técnico no puede revelar.
- **Procesamiento de documentos escaneados:** Módulo OCR para normativa histórica disponible solo como imagen.
- **Expansión regional:** La arquitectura modular permite adaptación a otros países latinoamericanos con ajustes mínimos de corpus y prompts.

Referencias Bibliográficas

- Asamblea Legislativa Plurinacional. (2024). Gaceta Oficial del Estado Plurinacional de Bolivia [Consultado en 2026].
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2.^a ed.). Addison-Wesley.
- CEPAL. (2021). Transformación digital y gobierno abierto en América Latina [Consultado en 2026].
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv:2402.03216*.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*.
- El País. (2025). Delitos contra la propiedad se incrementaron en 2023 [Consultado en 2026].
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*.
- Estado Plurinacional de Bolivia. (1972). Código Penal Boliviano — Ley N° 10426 [Con modificaciones vigentes].
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine].
- Instituto Nacional de Estadística. (2023). Encuesta de Victimización (EVIC) 2023: Resultados generales [Consultado en 2026].
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3.^a ed.). Pearson.

- LangChain, Inc. (2024). LangGraph: Build Stateful, Multi-Actor Applications with LLMs [Consultado en 2026].
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *arXiv preprint arXiv:2005.00661*.
- Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239).
- Neo4j, Inc. (2024). Neo4j Graph Database Platform [Consultado en 2026].
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982-3992.
- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases: New Opportunities for Connected Data* (2.^a ed.). O'Reilly Media.
- Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3.^a ed.). Prentice Hall.
- Susskind, R. (2019). *Online Courts and the Future of Justice*. Oxford University Press.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*, 30.